# Multithreading

1. Deskripsi
2. Membuat Thread (deskripsi Thread)
3. Kondisi Thread (pengkondisian Thread)
4. Siklus hidup Thread
5. Sinkronisasi
6. Package java.util.concurent
7. Contoh-contoh project

# Deskripsi

- Thread adalah sebuah unit eksekusi sekumpulan instruksi (dengan proses yg cukup lama) dalam sebuah class.
- Satu Thread dapat diwakili oleh beberapa objek.
- Namun, beberapa Thread tidak bisa diwakili satu objek.
- Dapat disederhanakan:
- **satu Thead diwakili satu Class/objek,**
- **dan satu Class/objek mewakili satu Thread.**
- Multithreading adalah suatu kemampuan yang memungkinkan beberapa Thread (memiliki proses yg cukup lama) dapat dijalankan secara bergantian/konkuren /simultan seakan bersamaan/paralel.
- Proses deskripsi **dilakukan di Class-class yg menjadi Thread.**

# Manfaat

- Manfaat aplikasi yang menggunakan multithreading yaitu agar thread-thread untuk suatu proses, dapat sharing kode program, data, dan sumber daya, secara lebih efisien.

# Membuat Thread dengan extends dari class Thread

- *public class NamaThread1 extends Thread {*
- *.....*
- *public void run() {*
- *.....*
- *}*
- *.....*
- *}*
- Class *NamaThread1* yang telah dibuat tidak dapat menjadi turunan dari class lainnya.

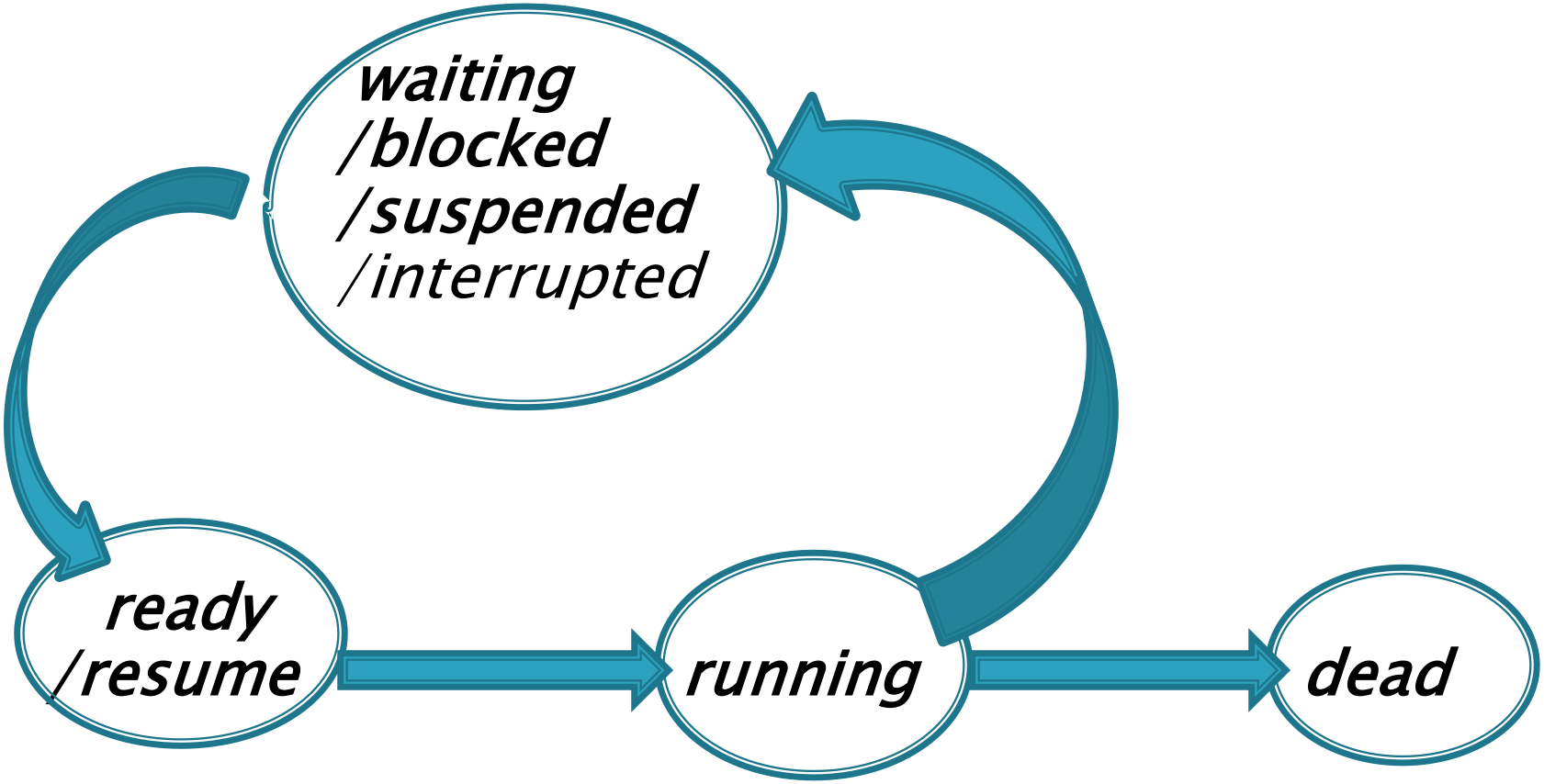
# Membuat Thread dengan implements dari interface Runnable

```
public class Bola extends Lingkaran, implements Runnable {
    Thread thread2;
    .........
    public void run() {
      .........
    }
  .........
}
```

- Cara ini menyebabkan lebih banyak pekerjaan karena object *Thread* harus dideklarasikan untuk memanggil method-method thread.
- Class NamaThread2 masih dapat menjadi turunan dari class lain.

# Kondisi Thread

- Thread dapat berada dalam 1 dari 4 kondisi berikut:




- Instruksi kode program untuk pengkondisian Thread **dilakukan di Main Class.**

# Kondisi Thread

- *Multitasking* adalah kemampuan komputer melakukan banyak tugas secara konkuren/bergantian. Pada multitasking, konkurensi melibatkan banyak proses.
- Karena *multithreading* perluasan dari multitasking, maka pada multithreading setiap proses melibatkan banyak thread.
- *Running* adalah kondisi saat thread sedang dieksekusi.
- *Ready* adalah kondisi saat thread siap dieksekusi namun belum ada kesempatan untuk dilakukan.
- *Resume* (mirip dengan ready) adalah kondisi saat thread siap dijalankan kembali setelah running dihentikan sementara waktu oleh **thread lain/sistem operasi/terhenti sendiri karena menunggu sumber daya.**
- *Suspended/waiting* adalah kondisi saat running suatu thread dihentikan sementara oleh **Sistem Operasi** agar thread lain gantian dieksekusi processor.
- *Blocked* (mirip dengan suspended/waiting) adalah kondisi saat thread menunggu **sumber daya** tersedia atau menunggu sebuah **event** terjadi.
- *Interrupted* adalah kondisi saat thread dihentikan oleh **thread lain** untuk menggunakan prosessor agar dieksekusi.

# Class dan Interface yang berkaitan dengan Multithreading

| Class/Interface | Deskripsi |
|---|---|
| Thread | Class untuk membuat Thread |
| ThreadDead | Class untuk membersihkan Thread yang eksekusinya telah berakhir. |
| ThreadGroup | Class untuk mengelompokkan Thread |
| Runnable | Interface untuk membuat Thread |

# Method-method standard yang berkaitan dengan Multithreading

| Nama method() | Deskripsi |
|---|---|
| start() | Dipanggil setiap kali thread dimulai. |
| init() | Gunanya sama dengan start() untuk menempatkan program inisialisasi. |
| run() | Berisi badan Thread. |
| wait() | Menunggu sampai thread lain memanggil method *notify() atau notifyAll().* |
| notify() | Mengaktifkan thread yang telah memanggil method *wait().* |
| notifyAll() | Mengaktifkan semua thread yg telah memanggil method *wait().* |
| stop() | Menghentikan thread dan ditempatkan di akhir badan thread. |
| join() | Thread yang sedang bekerja menunggu sampai method() yang memanggil selesai dieksekusi. |

# Siklus Hidup Thread

- Pertama kali thread diciptakan (**born**), selalu dalam kondisi **ready**.
- Method ***start()*** harus dipanggil untuk mengeksekusi method ***run()*** sehingga thread tersebut siap segera dieksekusi processor.
- Thread yang berada pada kondisi ***ready*** antri menunggu giliran untuk dieksekusi, penjadwal akan melakukan *dispatching* (menjadwal dan memilih) suatu thread sehingga thread tersebut transisi dari kondisi **ready** ke **running** (processor sedang mengeksekusi thread tersebut).

# Siklus Hidup Thread

- Jika saat eksekusi thread memerlukan kondisi tertentu untuk dipenuhi atau karena diinterupsi, maka thread tersebut dapat memanggil method **wait()** atau **sleep()** agar thread dapat transisi dari kondisi **running** ke kondisi **waiting** (menunggu sampai masuk kembali ke kondisi resume).
- Thread transisi dari kondisi **running** ke kondisi **dead** ketika method ***run()*** selesai dilaksanakan atau diakhiri karena terjadi exception, selanjutnya dihapus dari memori aktif.

# Sinkronisasi

- Thread-thread yang berjalan bersama-sama kadang-kadang membutuhkan sumber daya atau method dari luar, untuk berkomunikasi satu dengan yang lain agar saling mengetahui kondisi dan aktifitas mereka, maka diperlukan sinkronisasi.
- Java memperbolehkan penguncian terhadap sebuah objek termasuk method-methodnya dengan menggunakan monitor, sehingga dapat dipastikan bahwa hanya satu thread yang mendapatkan hak akses ke dalam suatu method.
- Dengan kata lain, sekali objek tersebut dimonitor, maka akan dipastikan bahwa tidak ada thread yang akan mengakses objek yang sama.

# Sinkronisasi

- Thread-thread yang berjalan bersama-sama terkadang memerlukan sumber daya seperti processeor atau I/O device atau data atau method dari luar agar dapat diakses bersama, sehingga perlu saling berkomunikasi agar saling mengetahui kondisi dan aktifitas mereka, maka diperlukan sinkronisasi.
- Jika sinkronisasi diterapkan pada suatu method, dipastikan bahwa seluruh kode di dalam method tersebut dieksekusi tanpa diinterupsi oleh yang lain.
- Sinkronisasi pada suatu method adalah suatu upaya agar kode program tertentu dijalankan secara sekuensial sehingga kode tersebut tidak akan dijalankan oleh thread lain dalam waktu yang bersamaan.
- Format umum sinkronisasi :
- *synchronized (<object>) {*
- *//statements yang akan disinkronisasikan*
- *}*

# package java.util.concurent

- Hanya dua interface yang dibahas pada subbab ini yaitu *Executer* dan *Callable*.
- Sebelumnya diketahui bahwa untuk mengeksekusi thread pada class yang mengimplementasikan *interface runnable* menggunakan instruksi :
- **new Thread(<aRunnableObject>).start();**
- Permasalahan dengan teknik ini membutuhkan pengaturan agar tidak terjadi penumpukan thread di suatu tempat, jika terjadi maka dapat mengakibatkan error pada memori.

# package java.util.concurent

- Solusi yang ditawarkan yaitu menggunakan *pooling thread* dengan skema yang baik, walaupun solusi ini memiliki kekurangan yaitu sulit untuk membatalkan atau mematikan sebuah thread.
- Sehingga untuk mengeksekusi thread pada class yang mengimplementasikan *interface runnable* menggunakan instruksi : ***<ObjekEksekutor>.execute(<ObjekRunnable>);***
- Mengggunakan interface Executor :
- ***Executor <namaEksekutor> = <ObjekEksekutor>;***
- ***<namaEksekutor>.execute(new <Task1>());***
- ***<namaEksekutor>.execute(new <Task2>());.***

# Interface Executor

- Membuat sebuah object dari tipe Executor tidak dapat dilakukan, namun dapat membuat sebuah class yang mengimplementasikan interface ini, dapat menggunakan method ***factory()*** yang telah disediakan class ***Executor***, untuk mengelola *thread pool* yang sederhana.
- Interface ini juga mengontrol eksekusi, menyelesaikan tugas-tugas yang *Runnable* serta menghentikan threads dengan ***executor.shutdown()***.

# Interface Collable

Sebelumnya untuk mendapatkan hasil dari sebuah tugas yang *Runnable* dilakukan dengan cara:

```
public class MyRunnable implements Runnable {
    private int result = 0;
    public void run() {
        ...
        result = someValue;
    }
    /* attribut result terlindung kode lain yang mengakses class ini */
    public int getResult() {
        return result;
    }
}
```

# Interface Collable

Untuk mendapatkan hasil dari sebuah tugas yang *Runnable* dilakukan dengan cara:

```
import java.util.concurrent.*;
public class MyCallable implements Callable {
   public Integer call() throws java.io.IOException {
        ...
        return someValue;
   }
}
```

# Contoh1 project

pCountDown

# Contoh project

pThread:
Mobil.java
AplyThreadMobil.java
AplyThreadMobil2.java

Pada AplyThreadMobil.java walaupun thread mobil1 dijalankan terlebih dahulu kemudian dijalankan thread mobil2 namun eksekusi dilaksanakan TIDAK menuntaskan dulu thread mobil1sampai selesai baru mobil2, jadi eksekusi dilaksanakan secara bergantian.

Pada AplyThreadMobil2.java ditambahkan kode program untuk mengecek apakah suatu thread sudah tuntas dieksekusi atau belum.

# pThread

PrintNameThread1.java

TestThread1.java

- Pada pasangan class yang pertama ini thread A, B, C, dan D, dieksekusi terlebih dahulu sampai tuntas setelah itu eksekusi thread E, F, G, dan H.

PrintNameThread2.java

TestThread2.java

- Pada awalnya eksekusi dilaksanakan urut thread A sampai selesai kemudian thread B, …. dst maka dengan method join() maka eksekusi tidak harus urut dan tuntas terlebih dahulu. Penggunaan method join() menyebabkan thread yang sedang dieksekusi menungggu sampai thread yang memanggil selesai dieksekusi.

# Contoh project

pThread:

MyOtherThread.java

AplyMyOtherThread.java

Contoh ini pada prinsipnya sama dengan contoh-contoh sebelumnya, hanya cara pembuatan thread dengan implements ke interface Runnable.

# Contoh project

pThread:

PrintStringsThread1.java

TwoStrings1.java

TestThread3.java

Pada contoh ini dua string yang akan diprint belum disinkronisasi terhadap thread lain sehingga urutan eksekusi thread tidak menentu.

# Contoh project

pThread:

TwoStrings2.java

PrintStringsThread2.java

TestThread4.java

method print() pada class TwoStrings2 sudah menggunakan modifier synchronized, maka output class TestThread4 sudah mulai terurut namun masih per string.

# Contoh project

pThread:

TwoStrings3.java

PrintStringsThread3.java

TestThread5.java

Penggunaan modifier synchronized tidak lagi di class TwoStrings,java namun dipindah ke dalam method run() saat memanggil method print() di class PrintStringsThread3.java.

Output class TestThread5 sudah terurut namun masih belum sempurna tercetak tiap dua string.

# Contoh project

pThread:

Urutan eksekusi thread tidak hanya diatur menggunakan modifier synchronized namun juga dengan setting prioritas thread.

Pada class AplySinkron2.java thread Mobil2 dieksekusi terlebih dahulu karena memiliki prioritas lebih tinggi.

# Contoh project

pThread:

Pada class MultiThread2.java dicoba setting prioritas. Thread memiliki prioritas 1 s/d 10.

Prioritas 1 terendah dan 10 tertinggi.

# Contoh project

pProducerConsumer:
Producer.java
Consumer.java
SharedData.java
TestProducerConsumer.java
Producer2.java
Consumer2.java
SharedData2.java
TestProducerConsumer2.java